EV BATTERY MONITORING SYSTEM FOR EARLY FIRE DETECTION AND SAFETY

A project report submitted in partial fulfillment of requirements to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA

For the award of the degree of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

Under the esteemed guidance of

Mr. A.R.V.S GUPTA, M.Tech,
Assistant Professor, Department of ECE

Submitted By

M. Terisha Lakshmi Devi	(22A21A04B7)
G.Siva Manikanta	(22A21A0461)
G. Prasanna	(22A21A0481)
K.Lokesh	(23A25A0414)

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

SWARNANDHRA COLLEGE OF ENGINEERING & TECHNOLOGY

(AUTONOMOUS)

Accredited by NAAC with 'A' Grade - 3.02/4.00 CGPA and Accredited by NBA

Approved by A.I.C.T.E & Permanently Affiliated to JNTU Kakinada

Seetharampuram, Narsapur-534 280, West Godavari Dt., A.P

2022-2026

SWARNANDHRA COLLEGE OF ENGINEERING AND TECHNOLOGY(AUTONOMOUS)

Accredited by NAAC with 'A' Grade-3.02/4.00 CGPA and Accredited by NBA

Approved by A.I.C.T.E & permanently affiliated to JNTUK Kakinada

Seetharampuram, Narasapur-534280, West Godavari (Dt.), A.P

Department of

ELECTRONICS AND COMMUNICATION ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this project report "**EV BATTERY MONITORING SYSTEM FOR EARLY FIRE DETECTION AND SAFETY**" is the bonafide work of M. Terisha Lakshmi Devi (22A21A04B7), G. Siva Manikanta (22A21A0461), G. Prasanna (22A21A0481), K. Lokesh (23A25A0414) in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Electronics and Communication Engineering, JNTUK, Kakinada during the academic year **2022–2026.**

PROJECT GUIDE	**HEAD OF THE DEPARTMEN**
Mr. A.R.V.S.Gupta	Dr.B.Subrahmanyeswara Rao

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

A project is a team work that is supported by a large number of well-wishers without any exception to this, we would like to thank a few people who have helped us to convert our dream to reality.

We extend our heartfelt gratitude to the almighty with profound gratitude, respect and pride, we express our sincere thanks to the management of Swarnandhra College of Engineering & Technology(A) **Sri K. V Satyanarayana, Chairman, Sri. K. V. Swamy, Treasurer, and Sri. A. Sri Hari, Director** for providing necessary arrangements to carry out of this project.

We wish to express our gratitude to **Dr. S. SURESH KUMAR, Principal** of **Swarnandhra college of engineering and Technology(A), Seetharampuram,** for giving permission to carry out the project.

We would like to express my sincere thanks to **Dr. B. SUBRAHMANYESWARA RAO, M.Tech, Ph.D Professor & Head of the Department of ECE** for offering his sincere support throughout the project work.

Our deep gratitude to and internal Guide **Mr.A.R.V.S.GUPTA, M.Tech., Assistent Professor, Project Guide**, **Department of ECE**. We thank him for dedication, guidance, council and keen interest at every stage of our project.

We would like to extend thanks to all the other staff member both teaching and non-teaching, for their timely help. Last but not least we owe our sincere thanks to our parents and all our friends, without whose encouragement our achievement would not have been possible.

PROJECT ASSOCIATES:

M. Terisha Lakshmi Devi (22A21A04B7)
G.Siva Manikanta (22A21A0461)
G. Prasanna (22A21A0481)
K.Lokesh (23A25A0414)

DECLARATION

I certify that

a. The Major project work has been done under the guidance of my supervisor.

b. The work has not been submitted to any other university for the award of any degree or diploma.

c. The guidelines of the university are followed in writing the thesis.

Date:

Place:

Reg. No	Name of the Student	Signature
22A21A04B7	M.TERISHA LAKSHMI DEVI	
22A21A0461	G.SIVA MANIKANTA	
22A21A0481	G.PRASANNA	
23A25A0411	K.LOKESH	

ABSTRACT

The Electric Vehicle (EV) Battery Monitoring System for Early Fire Detection and Safety is designed to enhance the safety of EV batteries by continuously monitoring critical environmental parameters such as temperature, gas leakage, and flame presence. The system utilizes sensors like DHT11, MQ2 gas sensor, and an IR flame sensor interfaced with an ESP32 microcontroller to detect abnormal conditions that may lead to thermal runaway or fire hazards. Real-time data is processed and displayed on an OLED screen, while a buzzer alert system provides immediate notification in case of danger, ensuring quick preventive action.This project offers a cost-effective and reliable solution for improving battery safety in electric vehicles by enabling early detection of potential fire risks. The system operates efficiently without the need for complex cloud integration, making it simple and practical for implementation.

LIST OF CONTENTS

LIST OF FIGURES:

LIST OF TABLES:

CHAPTER-01
INTRODUCTION

CHAPTER-01
INTRODUCTION

1.1 INTRODUCTION TO EMBEDDED SYSTEMS

The microprocessor-based system is built for controlling a function or range of function-sand is not designed to be programmed by the end user in the same way a PC is defined as an embedded system. An embedded system is designed to perform one particular task albeit with different choices and options.

Embedded systems contain processing cores that are either microcontrollers or digital signal processors. Microcontrollers are generally known as "chips", which may itself be packaged with other microcontrollers in a hybrid system of Application Specific Integrated Circuit (ASIC). In general, input always comes from a detector or sensors in more specific words and meanwhile the output goes to the activator which may start or stop the operation of the machine or the operating system.

An embedded system is a combination of both hardware and software, each embedded system is unique and the hardware is highly specialized in the application domain. Hardware consists of processors, microcontrollers, IR sensors etc. On the other hand, Software is just like a brain of the whole embedded system as this consists of the programming languages used which makes hardware work. As a result, embedded systems programming can be a widely varying experience.

An embedded system is a combination of computer hardware and software, either fixed incapability or programmable, that is specifically designed for a particular kind of application device. Industrial machines, automobiles, medical equipment, vending machines and toys (as well as the more obvious cellular phone and PDA) are among the myriad possible hosts of an embedded system. Embedded systems that are programmable are provided with a programming interface, and e mbedded systems programming id specialized occupation.

On the other hand, the microcontroller is a single silicon chip consisting of all input, output and peripherals on it.

A single microcontroller has the following features:

- Arithmetic and logic unit.
- Memory for storing program.
- EEPROM for non-volatile and special function registers.

- Input/output ports.
- Analog to digital converter.
- Circuits.
- Serial communication ports.

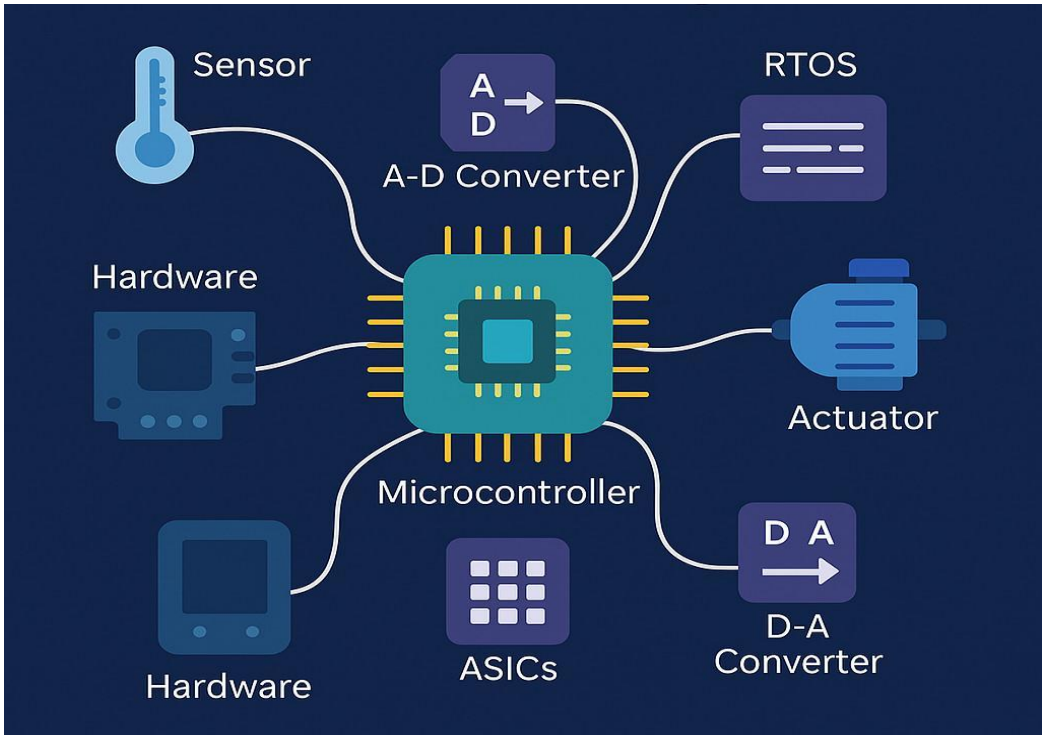


Fig1.1: Working Module of an Embedded Systems

1.2 EMBEDDED COMMUNICATION PROTOCOLS

Embedded systems use various communication protocols to enable data transfer between different components such as sensors, microcontrollers, and output devices. These protocols ensure reliable, fast, and efficient communication within the system as well as with external devices.

- **UART (Universal Asynchronous Receiver Transmitter)** – Simple serial communication used for data exchange between devices.
- **SPI (Serial Peripheral Interface)** – High-speed communication protocol used to connect microcontrollers with peripherals like sensors and displays.
- **I2C (Inter-Integrated Circuit)** – Two-wire communication protocol that allows multiple devices to communicate on the same bus.
- **CAN (Controller Area Network)** – Robust protocol mainly used in automotive and industrial applications for reliable communication.

- **USB (Universal Serial Bus)** – Used for data transfer and connection between embedded devices and computers.
- **Wi-Fi** – Enables wireless communication and is widely used in IoT-based embedded systems.
- **Bluetooth** – Used for short-range wireless communication between devices

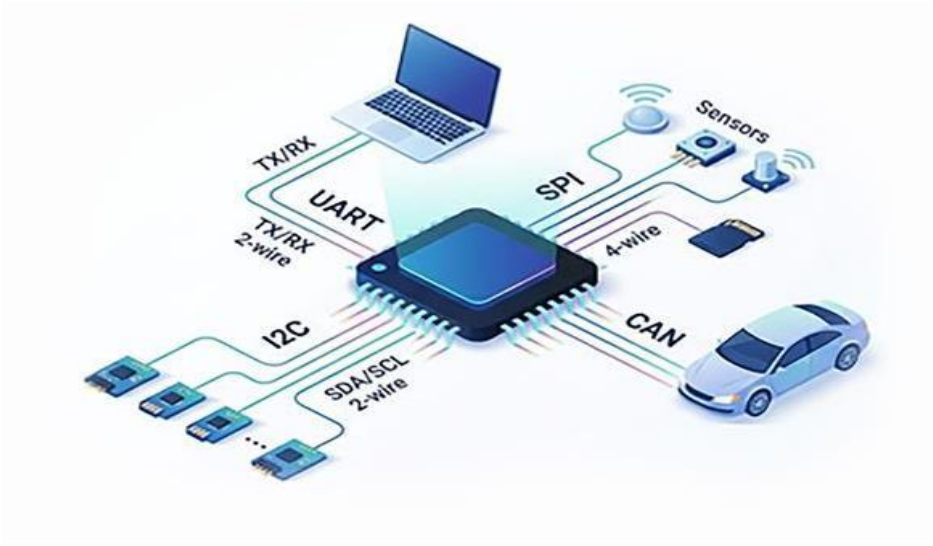


Fig 1.2 Embedded Communication Protocols

1.3 CHARACTERISTICS OF EMBEDDED SYSTEMS

- **Speed (bytes/sec):** Should be high speed.
- **Power (watts):** Low power dissipation.
- **Size and weight**: As far as possible small in size and low weight.
- **Accuracy (%error):** Must be very accurate.
- **Adaptability**: High adaptability and accessibility.
- **Reliability**: Must be reliable over a long period of time.

1.4 ADVANTAGES OF EMBEDDED SYSTEMS

- It is able to cover a wide variety of environments.
- likely to encore errors.
- Embedded System simplified hardware which reduces costs overall.
- Offers an enhanced performance.
- The embedded system is useful for mass production.
- The embedded system is highly reliable.
- It has very few interconnections.

1.5 DISADVANTAGES OF EMBEDDED SYSTEMS

- To develop an embedded system requires high development effort.
  It takes a long time to market.
- Embedded systems do a very specific task, so it can't be programmed to do different things.
- Embedded systems offer very limited resources for memory.
- It doesn't offer any technological improvement.

1.6 COMMON FEATURES OF EMBEDDED SYSTEMS

- It is designed to do a specific task, like general purpose computers.
- It does not see related computers; there may not be a key board or full monitor.
- More embedded systems must be able to do things in real time in a short amount of time.
- It starts very fast people don't want a minute time for their vehicle start or emergency devices to start.
- Embedded systems may use a special operating system that helps meet these requirements called RTOS or real time operating system.
- The program instructions written for embedded systems are referred to as flash memory chips and firmware. They run with limited computer hardware resources: small screen and keyboards, little memory.
- This system is not always a standalone device. Periodically they are built as a set like a various parts of the car: the throttle control, pollution controller, the radio etc.

1.7 APPLICATIONS OF EMBEDDED SYSTEMS

Embedded systems are widely used in various applications due to their ability to perform specific tasks efficiently and in real time. They play a vital role in modern technology by enabling automation, accuracy, and smart functionality in different fields. The following are some major applications of embedded systems:

- Embedded systems are used in agriculture for smart farming and automated irrigation systems.
- In the automotive sector, they are used for navigation, driver assistance, and safety features.
- Industrial applications include machine control, process automation, and monitoring systems.

- In security systems, they are used for surveillance, biometric access, and alarm systems.
- Infrastructure applications include smart cities and traffic management systems.
- Consumer electronics include home appliances and entertainment systems like smart TVs.
- In healthcare, they are used for patient monitoring and medical instruments.
- Wearable devices such as smartwatches and fitness bands use embedded systems.
- Safety systems include emergency alert systems and fire alarms.
- Embedded systems are also used in toys like drones and wireless remote control devices.

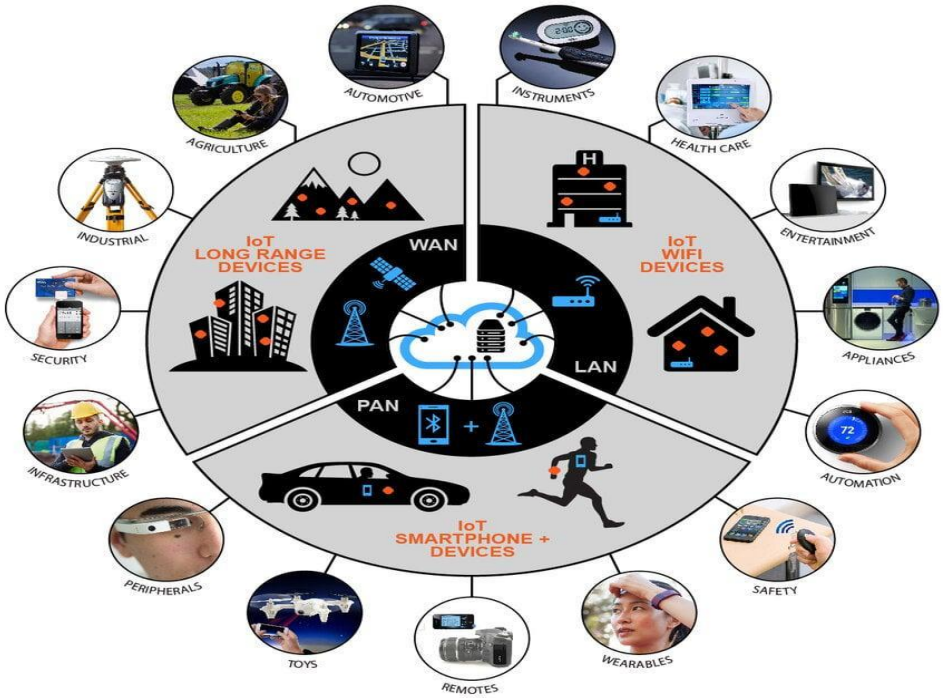


Fig 1.3: Applications of Embedded Systems

CHAPTER-02

LITERATURE SURVEY

CHAPTER-02
LITERATURE SURVEY

2.1 EXISTING SYSTEM

Current electric vehicles (EVs), battery monitoring and safety are primarily handled by a Battery Management System (BMS). The BMS is responsible for monitoring important parameters such as voltage, current, temperature, and state of charge of the battery. It ensures proper charging and discharging, balances individual cells, and protects the battery from overcharging, deep discharge, and short circuits.

Most existing systems rely heavily on electrical parameter monitoring. Temperature sensors are commonly used to detect overheating, and once the temperature exceeds a predefined threshold, the system may trigger cooling mechanisms or shut down the battery. Some advanced systems also include thermal management using cooling fans or liquid cooling.

However, these systems mainly focus on internal battery conditions and often lack early fire detection mechanisms. They do not effectively detect external factors such as:

- Gas leakage from battery cells
- Smoke formation
- Open flames

As a result, in situations like thermal runaway, where battery temperature rises rapidly and leads to fire, the existing systems may respond too late. Many EV fire accidents occur due to delayed detection and lack of multi-sensor safety mechanisms.

Additionally, traditional systems are:

- Expensive and complex
- Limited to built-in vehicle protection
- Not designed for real-time multi-condition hazard detection

Fig 2.1: Existing System

2.2 KEY STEPS IN PROCESS

- **Power Supply Initialization**

The system is powered on, and all components such as ESP32, sensors, OLED display, and buzzer receive the required voltage supply.

- **Sensor Activation**

Sensors including DHT11 (temperature & humidity), MQ-2 (gas sensor), and flame sensor start monitoring the battery environment continuously.

- **Data Acquisition**

All sensors collect real-time data such as temperature rise, gas leakage, and flame presence from the EV battery area.

- **Data Processing (Controller)**

The ESP32 processes the sensor data using programmed logic and compares it with predefined threshold values.

- **Abnormal Condition Detection**

If temperature exceeds safe limits, gas concentration increases, or flame is detected, the system identifies it as a potential fire hazard.

- **Decision Making**

Based on sensor inputs:

Normal condition → System continues monitoring

Abnormal condition → Trigger safety actions

- **Alert Indication**

Buzzer and/or LED are activated to provide immediate warning to the user about potential fire risk.

- **Display Output**

OLED display shows real-time values (temperature, gas level, fire status) and warning messages.

- **Safety Action**

System may stop battery operation (if integrated) or alert the user to take preventive action.

- **Continuous Monitoring Cycle**

The system continuously repeats the monitoring process to ensure real-timsafetyand early fire detection.

2.3 LIMITATIONS OF EXISTING SYSTEM

- They do not detect gas leakage, which is an early indication of battery failure.
- Flame detection is not included, so fire is identified only at a later stage.
- The system shows delayed response during thermal runaway, increasing risk.
- Lack of multi-sensor integration reduces reliability of hazard detection.
- No proper early warning or alert system for users in critical conditions.
- Unable to detect smoke or harmful gases released from battery cells.
- Response time is slow, as actions are triggered only after threshold limits are crossed.
- Existing systems mainly monitor voltage, current, and temperature only.
- Most systems are focused on internal battery parameters and ignore external factors.
- Limited real-time monitoring visibility for users in low-cost systems.
- Difficult to upgrade or modify existing systems with additional safety features.
- High cost and complexity make implementation less flexible.
- Lack of visual and audible alerts (like buzzer/display) in many systems.
- Not suitable for early-stage fire prevention, only for damage control.
- Reduced effectiveness in extreme environmental conditions.
- Dependency on predefined thresholds, which may not always be accurate.

CHAPTER-03

PROPOSED PROJECT

CHAPTER-03

PROPOSED PROJECT

3.1 INTRODUCTION TO PROJECT

Embedded systems play a vital role in modern technology by enabling automation and intelligent control across a wide range of applications. An embedded system is a combination of hardware and software designed to perform specific tasks efficiently and reliably. In recent years, the increasing demand for smart and safe systems has led to significant advancements in microcontrollers and sensor technologies. These developments have made it possible to design systems that can monitor environmental conditions in real time and respond to potential hazards effectively.

With the rapid growth of electric vehicles (EVs), ensuring battery safety has become a major concern. EV batteries, particularly lithium-ion batteries, are highly efficient but also sensitive to factors such as temperature variations, gas leakage, and internal faults. If these conditions are not monitored properly, they can lead to dangerous situations such as overheating, fire accidents, and explosions. Therefore, there is a strong need for a reliable system that can detect such conditions at an early stage and provide timely alerts.

The project focuses on the design and implementation of an EV Battery Monitoring System for Early Fire Detection and Safety. The main objective of this system is to continuously monitor critical parameters related to battery safety and to detect potential fire hazards at an early stage without human intervention. The system is capable of identifying abnormal conditions such as increased temperature, presence of harmful gases, and detection of flame, thereby improving the overall safety of the battery system.

The core component used in this project is the ESP32 microcontroller, which offers high processing speed, low power consumption, and built-in communication capabilities. The system integrates multiple sensors to achieve efficient monitoring. The DHT11 temperature sensor is used to measure the surrounding temperature of the battery, ensuring that overheating conditions are detected. The MQ2 gas sensor is employed to identify the presence of flammable gases or smoke, which are early indicators of battery failure. Additionally, the IR flame sensor is used to detect fire at an initial stage.

The data collected from these sensors is continuously processed by the ESP32 microcontroller. Based on the sensor readings, the system analyzes the current condition and determines whether it is safe or hazardous. If any abnormal condition is detected, such as high temperature, gas leakage, or flame presence, the system immediately activates an alert mechanism.

A buzzer is used to provide an audible warning, while an OLED display (128×64) is used to show real-time information about the system status.

This project demonstrates the effective integration of hardware components and software programming to develop a smart and reliable safety system. It is a cost-effective and efficient solution that can be used to enhance battery safety in electric vehicles. Furthermore, the system can be extended and applied in other areas where fire detection and safety monitoring are critical.

Overall, this project provides a practical understanding of embedded systems, sensor integration, and real-time monitoring, making it highly relevant in today's technology-driven world

3.2 OBJECTIVES

- To design and develop an EV battery monitoring system for early fire detection and safety.
- To continuously monitor temperature using DHT11 sensor to detect overheating conditions.
- To detect the presence of flammable gases or smoke using MQ2 gas sensor.
- To identify fire at an early stage using IR flame sensor.
- To process and analyze sensor data using the ESP32 microcontroller.
- To provide real-time monitoring of system parameters.
- To generate immediate alerts using a buzzer during abnormal conditions.
- To display system status and sensor readings on an OLED display (128×64).
- To ensure quick response and prevention of fire hazards in EV batteries.
- To develop a cost-effective and reliable safety system.
- To improve overall battery safety and user awareness.
- To implement a multi-sensor based monitoring system for improved accuracy.
- To detect early warning signs before critical failure occurs.
- To ensure fast and reliable response during emergency conditions.
- To minimize the risk of battery damage and fire accidents.

3.3 BLOCK DIAGRAM

The overall system architecture is represented in the block diagram below. The ESP32 microcontroller acts as the central hub, receiving input from the DHT11 sensor, MQ-2, IR flame sensor and sending control signals OLED indicators, and buzzer module.

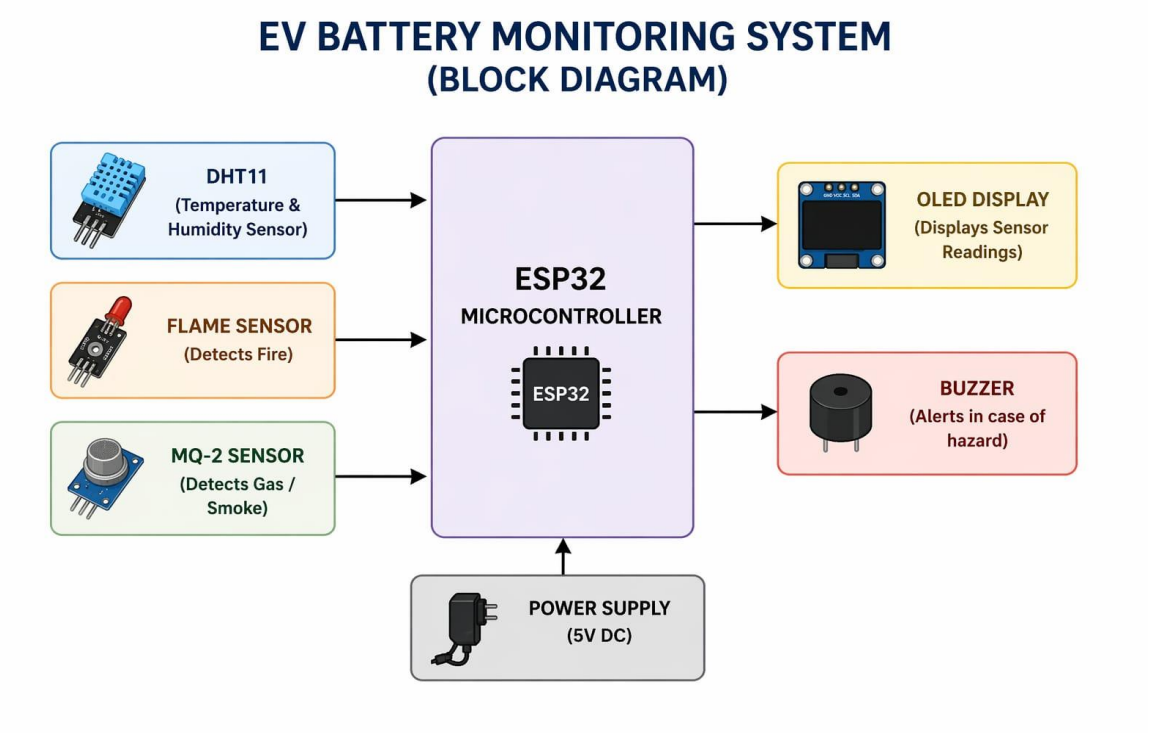


Fig 3.3 Block Diagram

The block diagram of the EV Battery Monitoring System for Early Fire Detection and Safety illustrates the overall structure and working of the system. It mainly consists of input sensors, a processing unit, and output devices. The DHT11 temperature sensor, MQ2 gas sensor, and IR flame sensor are used as input components to continuously monitor temperature, gas leakage, and fire conditions around the battery. These sensors send their data to the ESP32 microcontroller, which acts as the central processing unit of the system.

The ESP32 processes the sensor data in real time and checks whether the conditions are normal or abnormal. If any unsafe condition such as high temperature, presence of gas, or flame detection is identified, the system immediately responds by activating the output devices.

3.4 SCHEMATIC DIAGRAM

The schematic diagram of the proposed EV battery monitoring system is centered around the ESP32 microcontroller, which acts as the main processing unit. The system integrates multiple sensors such as the DHT11 sensor for temperature and humidity measurement, the MQ-2 sensor for detecting hazardous gases, and the flame sensor for identifying fire presence. These sensors are connected to the GPIO pins of the ESP32, where analog and digital signals are continuously monitored.

The power supply is provided from the EV battery, which is stepped down to a suitable voltage level (5V/3.3V) using a DC-DC buck converter to safely operate the ESP32 and other components. The ESP32 processes real-time sensor data and compares it with predefined threshold values to detect abnormal conditions.

For output indication, an OLED display is connected via I2C communication (SDA and SCL pins) to show live sensor readings and system status. Additionally, a buzzer and LED are connected to GPIO pins to provide immediate alert signals when dangerous conditions such as high temperature, gas leakage, or fire are detected.

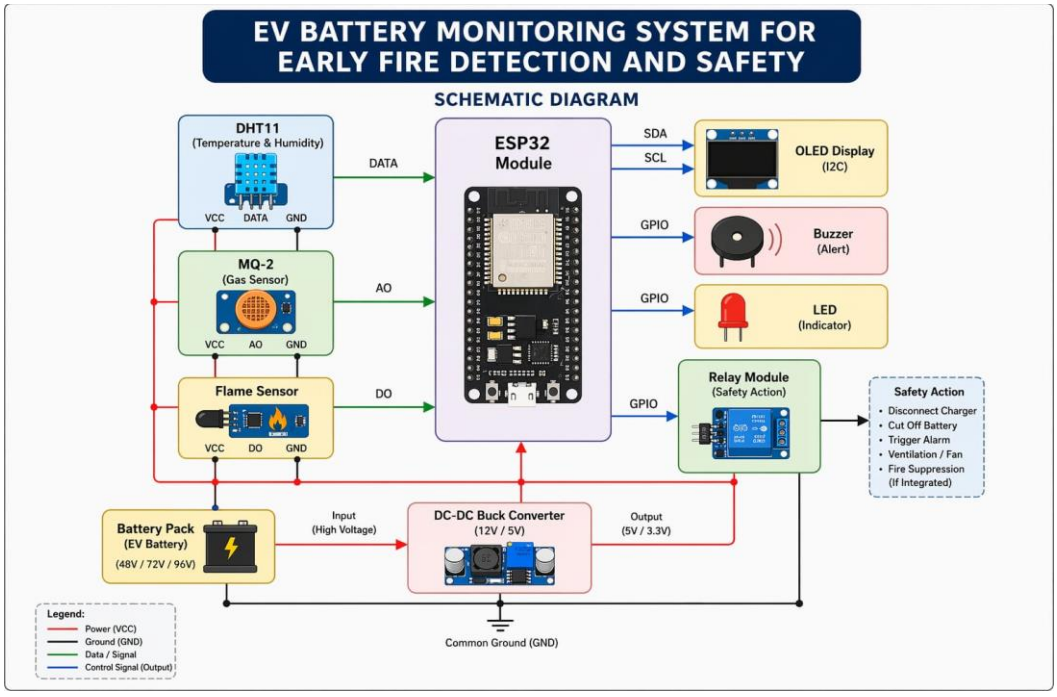


Fig 3.4 Schematic Diagram

3.5 WORKING OF THE SYSTEM

Step-1: Power ON the System

The EV battery pack or auxiliary power supply is switched ON, providing regulated power to all components such as microcontroller (ESP32), sensors, and communication modules.

Step-2: Initialization of ESP32

The ESP32 microcontroller initializes all connected modules like temperature sensors, gas/smoke sensors, voltage sensors, and communication interfaces (Wi-Fi/GSM).

The system starts program execution and sets safety thresholds.

Step-3: Sensor Data Acquisition

Temperature sensors continuously monitor battery cell temperature.

Gas/smoke sensors detect hazardous gases (like CO, electrolyte vapors).

Voltage and current sensors monitor battery health and abnormalities.

Step-4: Data Processing and Monitoring

The ESP32 reads real-time sensor data and processes it. It compares the values with predefined safe limits for temperature, gas levels, and voltage.

Step-5: Early Fire Detection

If temperature rises beyond a threshold or abnormal gas/smoke is detected, the system identifies a potential fire hazard.Rapid changes (thermal runaway conditions) are also detected.

Step-6: Decision Making by ESP32

If all parameters are within safe limits, the system continues normal monitoring.

- If danger is detected:The system triggers alerts.
- Decides emergency actions (shutdown/isolation).

Step-7: Alert and Notification System

Buzzer/LED alerts are activated locally.Notifications are sent to the user via mobile app, SMS, or IoT platform (Wi-Fi/GSM).

Step-8: Safety Control Actions

The system disconnects the battery using relays or contactors.

Activates cooling systems or fire suppression mechanisms if available.

Step-9: Continuous Monitoring

Even after alerting, the system keeps monitoring conditions to track changes and ensure safety.

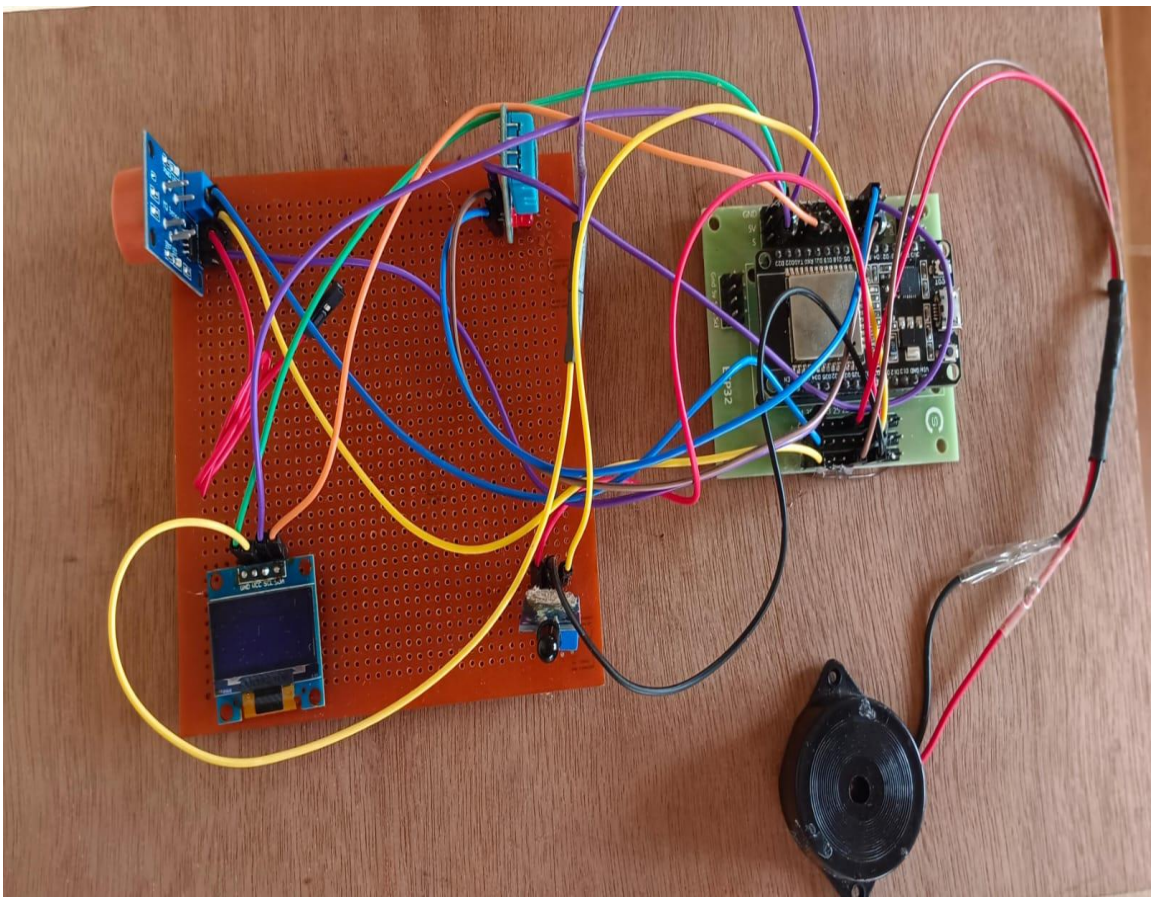


Fig 3.5: Project Kit

CHAPTER-04

HARDWARE COMPONENTS

CHAPTER-04
HARDWARE COMPONENTS

4.1 COMPONENTS

This section describes each hardware component used in the autonomous obstacle avoiding vehicle, detailing its functional role, operating principles, and integration within the overall system.

4.1.1 List of Components

The system modules for the ESP32-based EV battery monitoring system for early fire detection and safety consist of the following key components:

- ESP32 Microcontroller Module
- DHT11 Sensor (Temperature & Humidity Sensor)
- MQ-2 Gas Sensor
- Flame Sensor Module
- OLED Display (I2C)
- Buzzer Module
- EV Battery / Power Supply
- Connecting Wires
- PCB Board / Breadboard
- USB Programming Cable

4.2 ESP32 Microcontroller

The ESP32 is a powerful, low-cost, low-power system-on-chip (SoC) microcontroller developed by Espressif Systems, Shanghai. Introduced in 2016 as the successor to the highly successful ESP8266, the ESP32 represents a significant leap in capability, integrating a dual-core processor, substantially more memory, enhanced peripheral support, and built-in Bluetooth in addition to Wi-Fi.

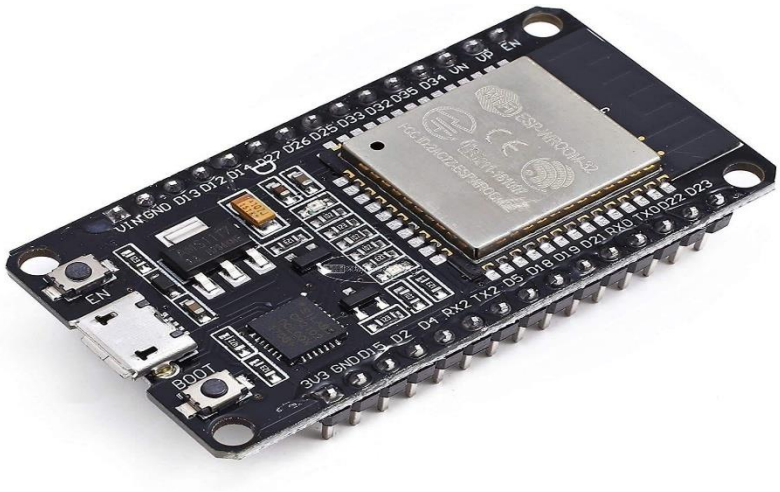


Fig 4.2: ESP32 Microcontroller WROOM Board

4.2.1 Architecture and Processing Core

The ESP32 is built around dual Xtensa LX6 32-bit RISC processor cores, each capable of operating independently at up to 240 MHz. This dual-core architecture allows time-critical tasks such as sensor polling and motor control to run on one core, while communication-intensive tasks like Wi-Fi and Telegram messaging run on the second core, preventing communication latency from interfering with real-time control loops.

The chip integrates 520 KB of internal SRAM for program execution and data storage, supplemented by external flash memory (typically 4 MB or more) for program storage. This memory capacity is substantially larger than classical 8-bit microcontrollers such as the ATmega328P (Arduino Uno), enabling more sophisticated algorithms, larger lookup tables, and more complex data structures.

4.2.2 Wireless Connectivity

The ESP32 integrates 802.11 b/g/n Wi-Fi supporting 2.4 GHz operation with data rates up to 150 Mbps, along with Bluetooth Classic (v4.2) and Bluetooth Low Energy (BLE). This dual wireless capability eliminates the need for external wireless modules, significantly reducing system cost and complexity. The Wi-Fi stack runs on a dedicated portion of the processor, minimizing interference with user application execution. WPA/WPA2 Enterprise security is supported, making the ESP32 suitable for deployment in corporate and institutional network environments.

Parameter	Specification
Processor	Dual-core Xtensa LX6, up to 240 MHz
SRAM	520 KB internal
Flash Memory	4 MB external (typical)
Wi-Fi Standard	802.11 b/g/n, 2.4 GHz
Bluetooth	v4.2 Classic + BLE
GPIO Pins	34 programmable
ADC	12-bit, 18 channels
PWM Channels	16
Operating Voltage	3.3 V (VIN accepts 5 V)
Operating Temp.	-40°C to +85°C

4.2.2 Peripheral Interfaces

4.2.3 Pin configuration

The ESP32 provides an extensive set of peripheral interfaces: 34 programmable GPIO pins, 12-bit SAR ADC with 18 channels, 8-bit DAC with 2 channels, 4 x SPI interfaces, 2 x I2C interfaces, 2 x I2S interfaces, 3 x UART interfaces, 16 x PWM channels, CAN 2.0 interface, and support for capacitive touch sensing and Hall effect sensing. This rich peripheral set enables direct interfacing with virtually any sensor or actuator without requiring external interface ICs.

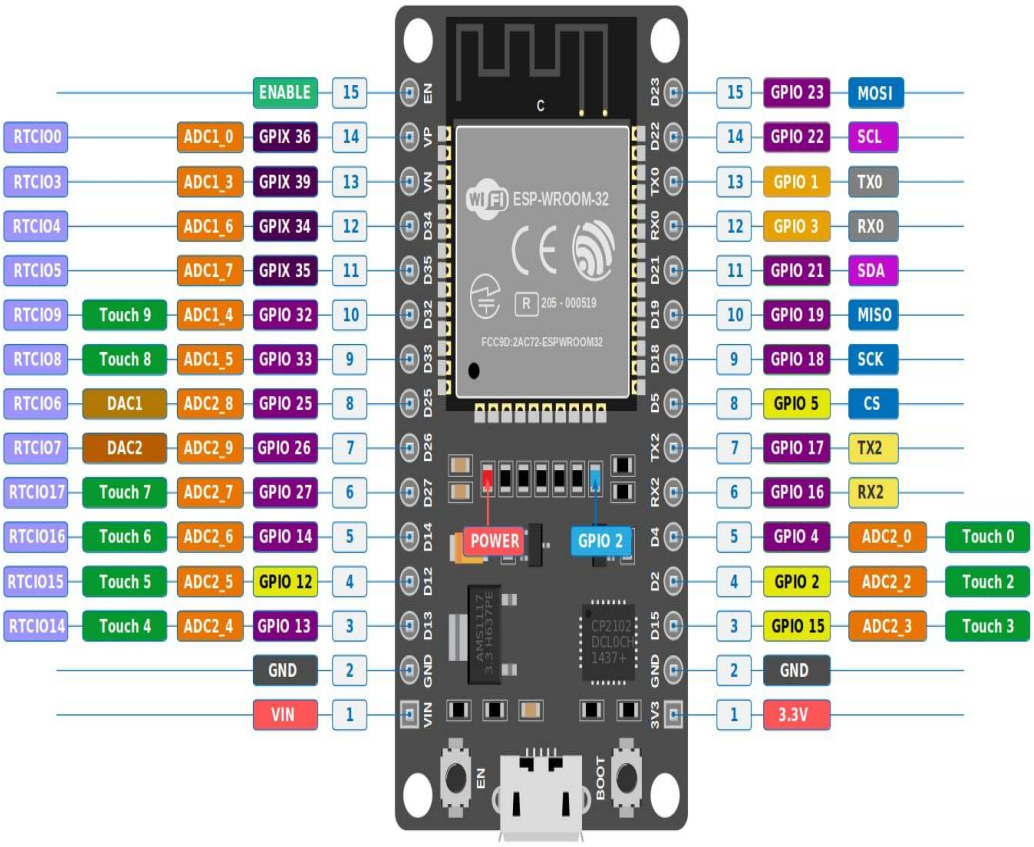


Fig 4.2.3 ESP32 Pin Configuration

4.3 DHT11 SESNOR (TEMPERATURE & HUMIDITY)

The DHT11 is a low-cost digital sensor used to measure temperature and humidity of the surrounding environment. It provides calibrated output in digital form, which can be easily read by the ESP32 microcontroller.

In this project, the DHT11 sensor is used to monitor the temperature near the EV battery. If the temperature rises beyond a safe limit, it helps in early detection of overheating conditions that may lead to fire hazards, thereby improving battery.

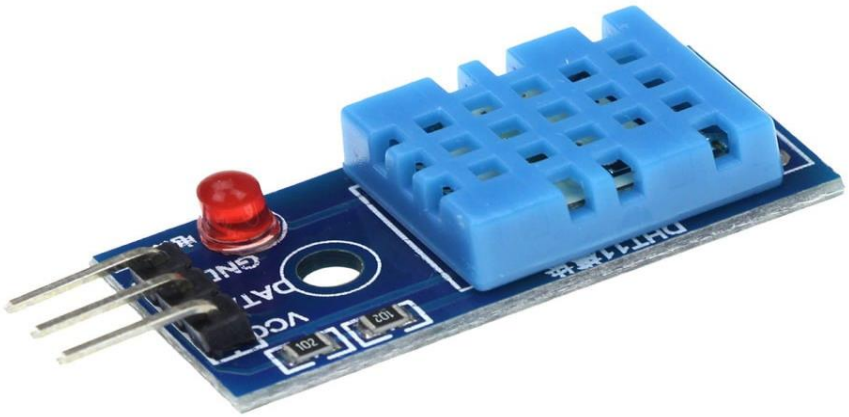


Fig 4.3:DHT11 sensor

4.3.1 Operating Principle

The DHT11 sensor works on the principle of sensing temperature and humidity using two internal components: a thermistor and a humidity sensing element.

Humidity Measurement:

It uses a capacitive humidity sensor. When the moisture level in the air changes, the capacitance of the sensor also changes. This variation is measured and converted into a humidity value.

Temperature Measurement:

It contains an NTC thermistor whose resistance changes with temperature. The sensor measures this change and converts it into temperature data.

Signal Processing:

An internal microcontroller processes both temperature and humidity values and converts them into a digital signal.

Data Transmission:

The processed data is sent to the ESP32 through a single data pin using a one-wire communication protocol.

Parameter	Specification
Operating Voltage	3.3v to 5v DC
Operating Current	0.3 mA
Temperature range	0°C to 50°C
Humidity Range	20% to 90% RH
Temperature Accuracy	±2°C
Humidity Accuracy	±5%RH
Sampling Rate	1Hz
Response Time	slow
Output signal	Digital(Single-wire)

4.3.2 Technical Specifications of DHT11 sensor

4.4 MQ-2 GAS SENSOR

The MQ-2 gas sensor is a widely used semiconductor sensor designed to detect combustible gases such as LPG, methane, hydrogen, smoke, and alcohol. It operates based on the change in resistance of a sensing material (SnO_2) when exposed to different gas concentrations. In clean air, the sensor has low conductivity, but when gas is present, its conductivity increases, producing a measurable change in output.

The sensor provides both analog and digital outputs, which can be interfaced with the ESP32 for continuous monitoring. In this project, the MQ-2 sensor is used to detect gas leakage or smoke around the EV battery. If the gas concentration exceeds a predefined threshold, it helps in early identification of hazardous conditions that may lead to fire, thereby improving the safety of the battery system.

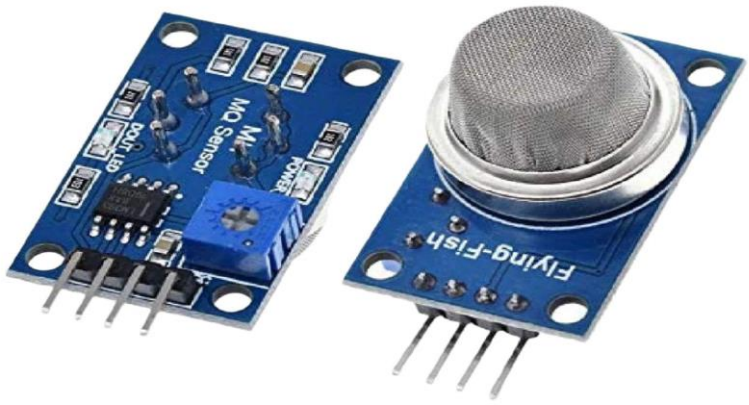


Fig 4.4 MQ-2 gas sensor

Parameter	Specification
Operating Voltage	5 V DC
Operating Current	15 mA (typical)
Detectable gases	LPG, Methane, Hydrogen, Somke , Alcohol
Gas Detection Range	200 ppm to 10000ppm
Humidity Range	5% to 95% RH
Output type	Analog(A0) and Digital(D0)
Preheat time	20 seconds(initial),24-48 hours

4.4.1 Technical Specifications of MQ-2 sensor

4.5 FLAME SENSOR

The flame sensor is a device used to detect the presence of fire or flame by sensing infrared (IR) light emitted by flames, typically in the wavelength range of 760 nm to 1100 nm. It consists of a photodiode or phototransistor that responds to IR radiation, along with a comparator circuit to generate output signals.The sensor provides both digital and analog outputs, which can be interfaced with the ESP32 microcontroller. When a flame is detected, the sensor output changes, allowing the controller to identify the presence of fire quickly.

In this project, the flame sensor is used to detect any fire occurrence near the EV battery. Upon detection, it triggers alerts such as a buzzer and LED, and can also activate safety mechanisms like a relay to prevent further damage, ensuring early fire detection and enhanced safety.

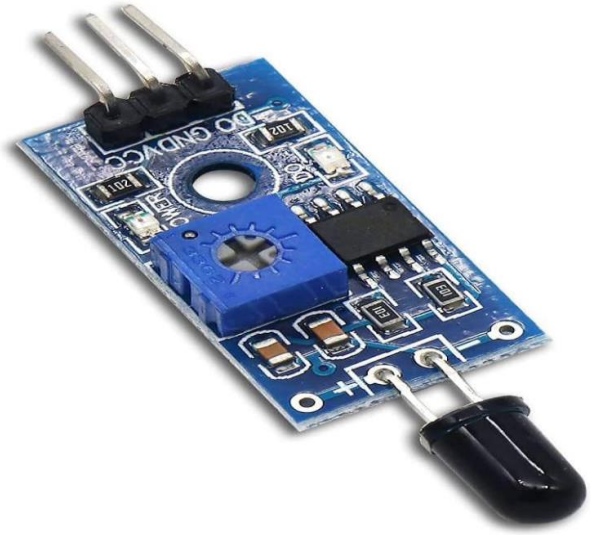


Fig 4.5 flame sensor

Parameter	Specification
Operating Voltage	3.3 V to 5 V DC
Operating Current	20 mA (typical)
Detection wavelength	760 nm to 1100nm
Detection Range	Upto 80 cm
Detection angle	60
Output type	Analog(A0) and Digital(D0)
Indicator LED	YES
Response time	Fast(few milliseconds)
Dimensions	Compact module

4.5.1 FLAME sensor Specifications

4.6 OLED DISPLAY

An OLED (Organic Light Emitting Diode) display is a compact and energy-efficient output device used to present real-time data in the EV battery monitoring system. Unlike traditional LCDs, OLED displays do not require a backlight, as each pixel emits its own light, resulting in higher contrast, better visibility, and lower power consumption. In this project, the OLED display is used to show important parameters such as temperature readings from the DHT11 sensor, gas levels from the MQ2 sensor, and fire detection status from the flame sensor. Its fast response time and clear display make it highly suitable for monitoring critical battery conditions. Additionally, the OLED module communicates easily with the ESP32 using I2C protocol, requiring fewer pins and simplifying the circuit design. Overall, the OLED display enhances the system by providing an efficient and user-friendly interface for real-time monitoring

Fig 4.6 OLED display

Working Principle

OLED consists of organic layers placed between two electrodes.

When current flows, these layers emit light directly.

Each pixel lights independently → better contrast and clarity.

Technical Specifications (Common 0.96" OLED)

Display Type: OLED

Driver IC: SSD1306

Resolution: 128 × 64 pixels

Communication: I2C (SDA, SCL)

Operating Voltage: 3.3V – 5V

Low power consumption

4.7 Buzzer and LED Indicators

An active piezoelectric buzzer is used to provide audible proximity alerts. Unlike passive buzzers that require an external oscillating signal, active buzzers contain an integrated oscillator circuit and produce a fixed-frequency tone when powered with a DC voltage between 3.3 V and 5 V. This simplicity makes them ideal for embedded alert applications where consistent audible feed-back is required without complex audio signal generation.

The buzzer is connected to a GPIO output pin of the ESP32. When the firmware sets the GPIO HIGH (3.3 V), the buzzer activates and produces its characteristic tone. When the GPIO is set LOW, the buzzer silences. The buzzer's power consumption is typically 30–40 mA, within the acceptable driving capability of the ESP32 GPIO when a transistor switch is used for current amplification.

Fig 4.7: Buzzer

4.8 POWER SUPPLY

The 3.7V power supply is a compact and efficient energy source commonly used in portable electronic and IoT systems. It is typically provided by a Lithium-ion (Li-ion) or Lithiumpolymer (Li-Po) battery, which offers high energy density, lightweight design, and

rechargeable capability.

In this project, the 3.7V power supply is used to power components such as the ESP32, Ultrasonic Sensor, and buzzer either directly or through voltage regulation. Since some components require stable 5V or 3.3V, voltage regulators or boost converters are used to ensure proper operation.

The battery can be recharged using a charging module (such as TP4056), making the system portable and suitable for real-time applications like public transport safety systems. The use of a 3.7V battery ensures low power consumption, reliability, and continuous operation during power output .Overall, the 3.7V power supply plays a crucial role in providing a stable and portable power source.

Fig 4.8:Power Supply

4.9 USB CABLE

A USB (Universal Serial Bus) cable is an essential component used for both power supply and data communication in the IoT safety system. It is used to connect the ESP32 microcontroller to a computer or power source. In this project, the USB cable supplies power to the ESP32, enabling it to operate and control all connected sensors and output devices. Additionally, it is used to upload program code from the computer to the ESP32 through the Arduino IDE. The cable also allows serial communication, which helps in debugging and monitoring system performance.

Typically, a USB cable consists of power lines (VCC and GND) and data lines (D+ and D−), ensuring reliable transmission of both electrical power and data signals. The commonly used cable for ESP32 is a USB-to-micro USB or USB Type-C cable, depending on the board model.

Overall, the USB cable plays a vital role in powering the system, programming the microcontroller, and enabling communication between the hardware and the computer during development and testing.

Fig 4.9: USB Cable

4.10 PCB BOARD

A Printed Circuit Board (PCB) is a mechanical and electrical platform used to connect electronic components using conductive copper tracks. It provides a compact, reliable, and organized way to assemble electronic circuits in embedded systems and industrial applications.

A PCB consists of an insulating substrate (such as fiberglass or epoxy) with copper layers laminated on it. These copper tracks replace manual wiring and provide permanent electrical connections between components like resistors, capacitors, ICs, microcontrollers, sensors, and power devices.

PCBs are designed using PCB design software and manufactured through etching processes. Based on complexity, they can be single-layer, double-layer, or multi-layer PCBs. This allows efficient routing of signals and power distribution in compact electronic systems.

In the proposed ESP32-based autonomous navigation robot, the PCB board is used to integrate all electronic components such as the ESP32 controller, motor driver circuit, sensor interfaces, power regulation circuit (buck converter), and LED indicators into a single compact board. This improves reliability, reduces wiring complexity, and enhances the overall performance of the system. The use of PCB ensures better durability, reduced signal interference, and a professional design suitable for real-time applications.

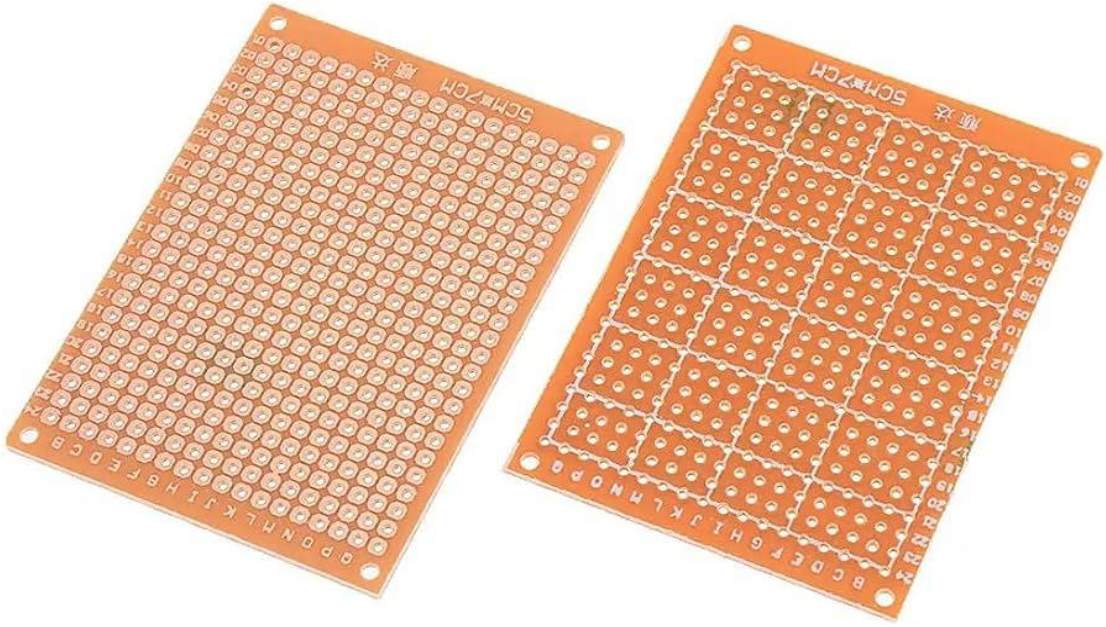


Fig 4.10: PRINTED CIRCUIT BOARD

4.11 CONNECTING WIRES

Connecting wires are essential electrical components used to establish electrical connections between different parts of an electronic circuit. They act as a medium for transmitting signals, power, and data between components such as microcontrollers, sensors, motors, power supplies, and other modules.

These wires are usually made of copper conductors because copper has high electrical conductivity and low resistance. They are covered with an insulating material (like PVC coating) to prevent short circuits and ensure safe operation of the circuit.

Connecting wires are available in different types such as male-to-male, male-to-female, and female-to-female jumper wires, especially useful in prototyping and embedded system projects.

In the proposed ESP32-based autonomous navigation system, connecting wires are used to interface various components like the ESP32 board, ultrasonic sensor, motor driver, LED indicators, buck converter, and power supply. They ensure proper signal flow between modules and help in building a complete working system.

Proper selection and neat arrangement of connecting wires are important to reduce noise, avoid loose connections, and improve the reliability of the overall circuit.

Fig 4.11: Connecting Wires

CHAPTER-05
WORKING FLOW

CHAPTER-05
WORKING FLOW

5.1 WORKING PROCEDURE STEP BY STEP

1.System Initialization:

When the system is powered ON, the ESP32 microcontroller initializes all connected hardware components including the DHT11 temperature sensor, MQ2 gas sensor, flame sensor, OLED display, and buzzer. During this stage, the system checks the proper functioning of each component and sets predefined threshold values for temperature, gas concentration, and flame detection. This ensures that the system starts in a stable and ready state for monitoring.

2. Sensor Activation:

After initialization, all sensors are activated simultaneously to begin continuous monitoring. Each sensor operates independently and starts collecting environmental data around the EV battery. This parallel sensing approach increases system efficiency and ensures that no critical parameter is missed during operation.

3. Temperature Monitoring:

The DHT11 sensor continuously measures the temperature of the battery environment. A gradual or sudden increase in temperature may indicate overheating, which is one of the primary causes of battery fires. The sensor sends real-time temperature data to the ESP32, enabling early detection of abnormal thermal conditions.

4. Gas Detection:

The MQ2 gas sensor detects the presence of combustible gases and smoke that may be released due to battery leakage or internal chemical reactions. The sensor outputs analog signals corresponding to gas concentration levels. An increase in gas levels beyond a safe limit indicates a potential hazard, prompting further action by the system.

5. Flame Detection:

The flame sensor is responsible for detecting fire by sensing infrared radiation emitted by flames. It provides a digital output when a flame is detected within its range. This sensor acts as a critical safety component by directly identifying fire presence in the early stages.

6. Data Acquisition:

The ESP32 continuously collects data from all sensors through its ADC (Analog-to-Digital Converter) and digital input pins. The analog signals from the MQ2 sensor are converted into digital values, while digital signals from the flame sensor are directly processed. This ensures accurate and real-time data collection from multiple sources.

7. Data Processing and Analysis:

The collected sensor data is processed by the ESP32 using programmed logic. The system compares real-time values with predefined threshold limits to determine whether the conditions are normal or abnormal. Filtering techniques may also be applied to reduce noise and improve accuracy.

8. Decision Making:

Based on the analysis, the ESP32 makes decisions regarding the safety status of the battery. If any parameter such as temperature, gas level, or flame detection exceeds the safe threshold, the system classifies it as a potential fire hazard. This decision-making process is fast and ensures immediate response.

9. Alert Generation:

Once a hazardous condition is detected, the system activates the buzzer to provide an audible warning. At the same time, the OLED display shows real-time sensor values along with alert messages such as “HIGH TEMPERATURE”, “GAS DETECTED”, or “FIRE ALERT”. This helps users quickly understand the situation and take necessary action.

The system operates continuously in a loop, ensuring uninterrupted monitoring of battery conditions. Even after detecting a hazard, the system keeps tracking sensor values .

CHAPTER-06

SOFTWARE IMPLEMENTATION

CHAPTER-06
SOFTWARE IMPLEMENTATION

6.1 INSTALLATION OF ESP32 IN ARDUINO IDE

6.1.1 Introduction to Arduino IDE for ESP32

The Arduino Integrated Development Environment (IDE) is the primary programming tool used in this project. The ESP32 board support is added to the Arduino IDE via the Boards Manager using Espressif's official board package URL. The IDE provides a code editor, compiler, and uploader in a single interface. To program the ESP32 with Arduino IDE: Install the Arduino IDE, add the ESP32 board URL in Preferences, install the ESP32 board package via Boards Manager, install required libraries (ESP32Servo, Adafruit SSD1306, Adafruit GFX), connect the ESP32 via USB, select the appropriate board (ESP32 Dev Module) and COM port, and upload the sketch.

FIG 6.1.1 USB CABLE USED FOR ESP32

STEP 1: INSTALLING ARDUINO IDE

FIG- 6.1.2 ARDUINO OFFICIAL WEBSITE – DOWNLOAD PAGE

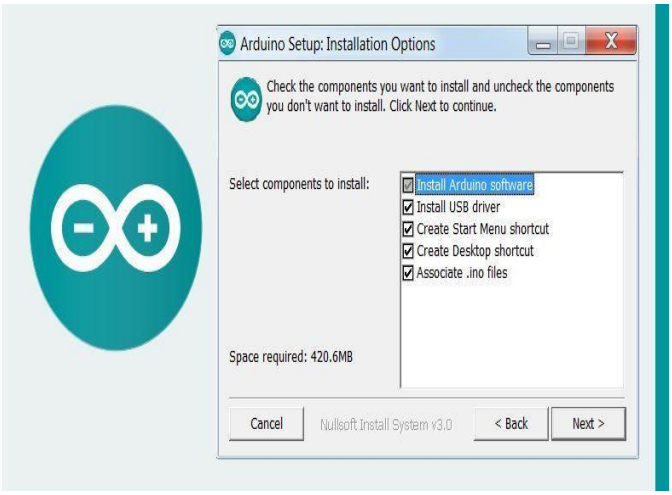


FIG- 6.1.3 ARDUINO IDE DOWNLOAD

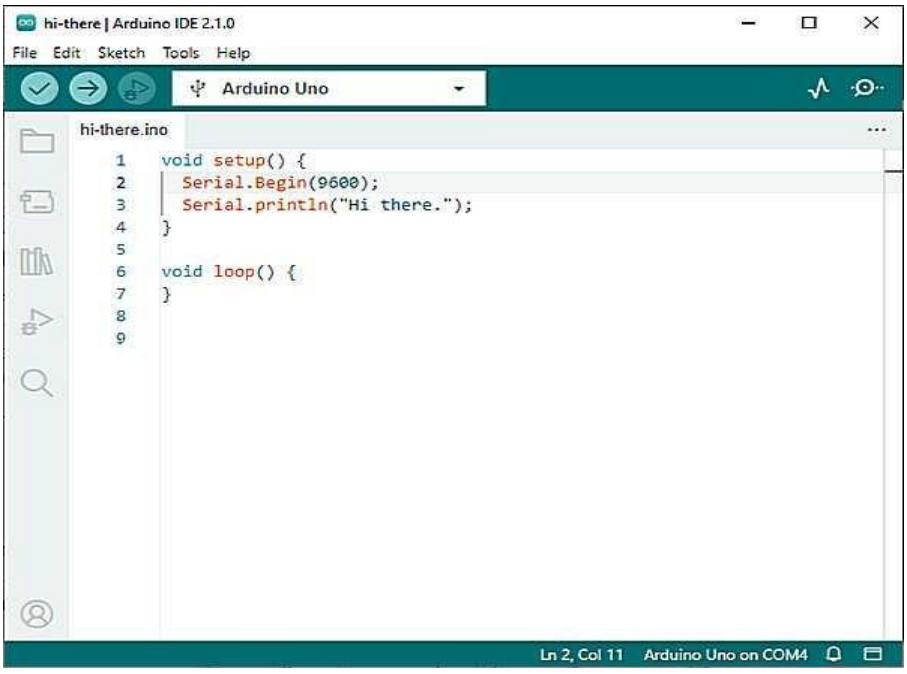


FIG 6.1.4 ARDUINO IDE INSTALLATION

PROCEDURE:

1. Go to the Arduino official website (https://www.arduino.cc/en/software)

2. Download the latest version of Arduino IDE for your OS (Windows/Mac/Linux)

3. Run the installer and complete the installation

4. Open Arduino IDE after installation is complete

STEP 2: INSTALLING ESP32 BOARD IN ARDUINO IDE

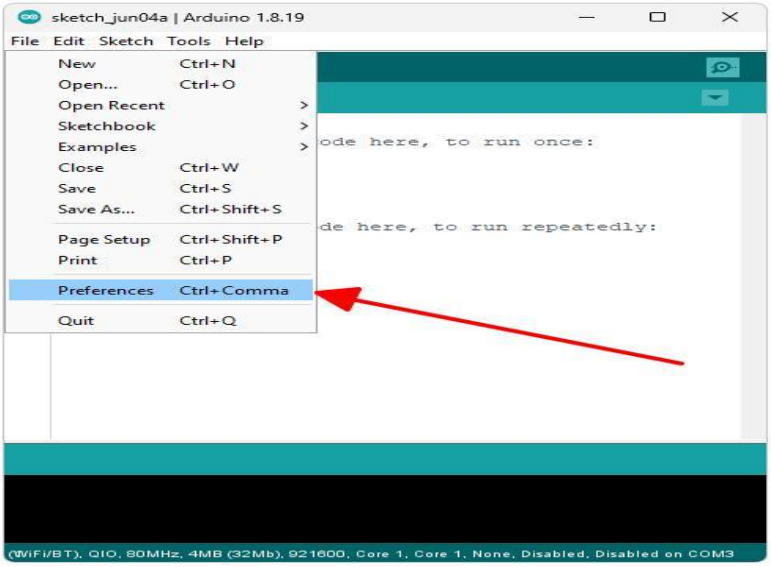


FIG 6.1.5 BOARD MANAGER - ESP32 SEARCH

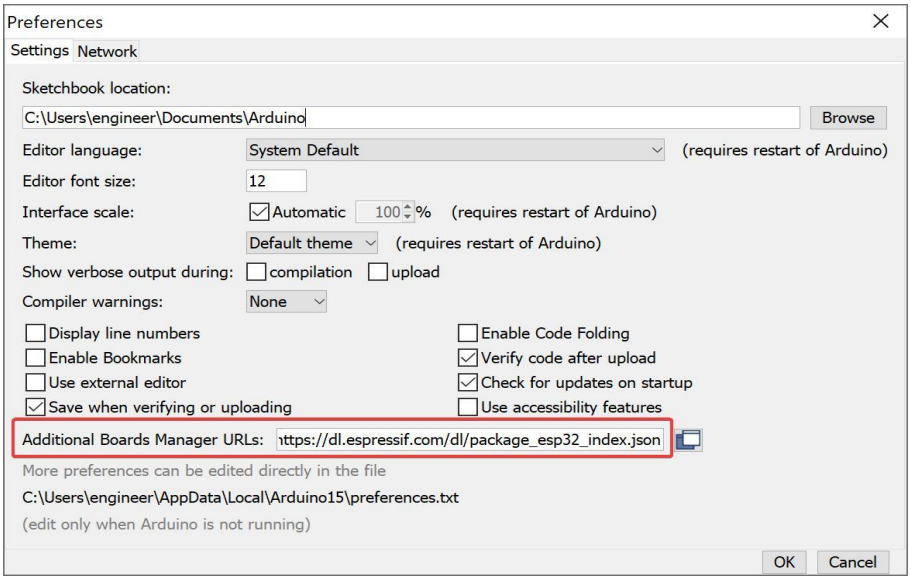


FIG 6.1.6 ADDING ESP32 BOARD URL IN PREFERENCES

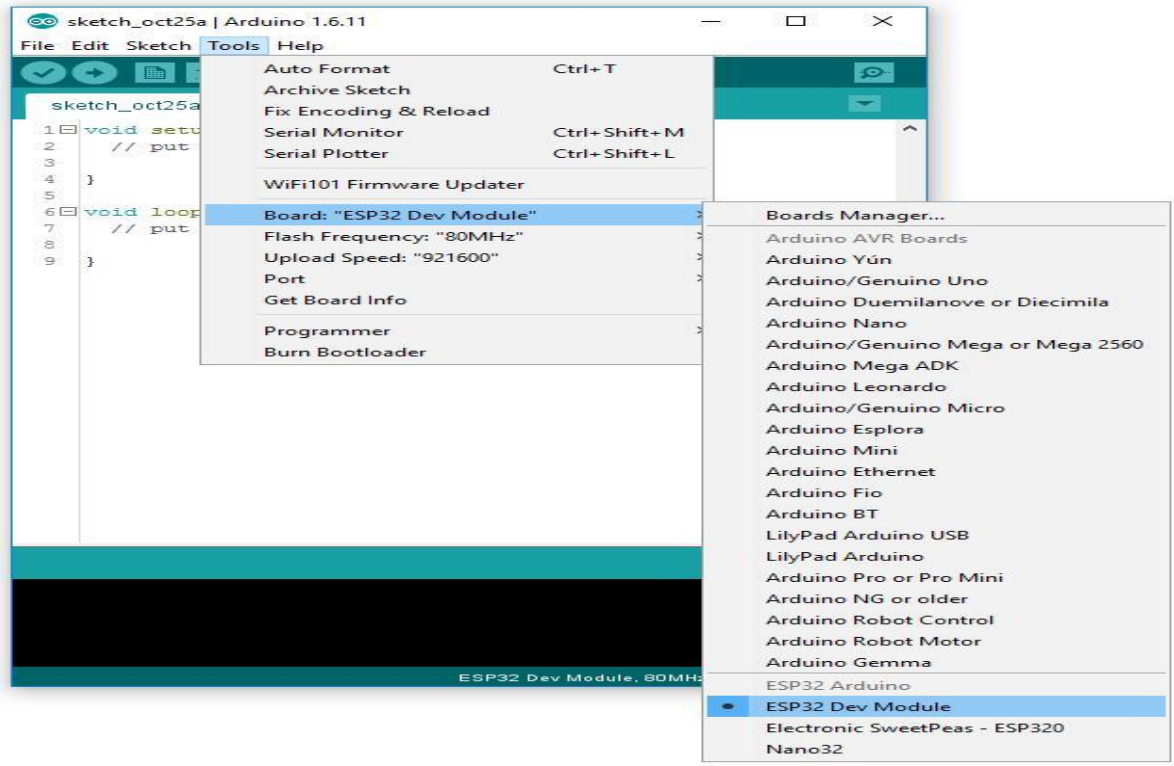


FIG 6.1.7 INSTALLING ESP32 BOARD PACKAGE

STEPS:

5. Open Arduino IDE
6. Go to File → Preferences
7. In“Additional Board Manager URLs” add:https://dl.espressif.com/dl/package_esp32_index.json
8. Go to Tools → Board → Boards Manager
9. Search “ESP32” in the search box
10. Select the package by Espressif Systems and click Install

STEP 3: INSTALLING REQUIRED LIBRARIES

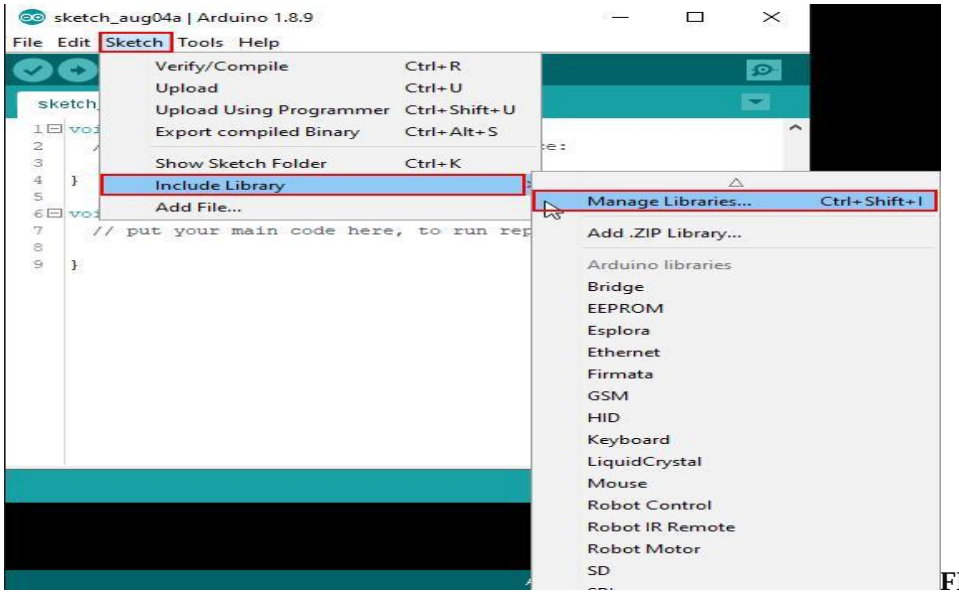


FIG 6.1.8 LIBRARY MANAGER - SEARCH LIBRARIES

FIG 6.1.9 INSTALLING LIBRARIES

Required Libraries: WiFi, WiFiClientSecure, UniversalTelegramBot, Wire, LiquidCrystal_I2C

Steps:

11. Go to Sketch → Include Library → Manage Libraries
12. Search each library by name in the search bar
13. Select the correct library and click Install

Step 4: Writing the Code

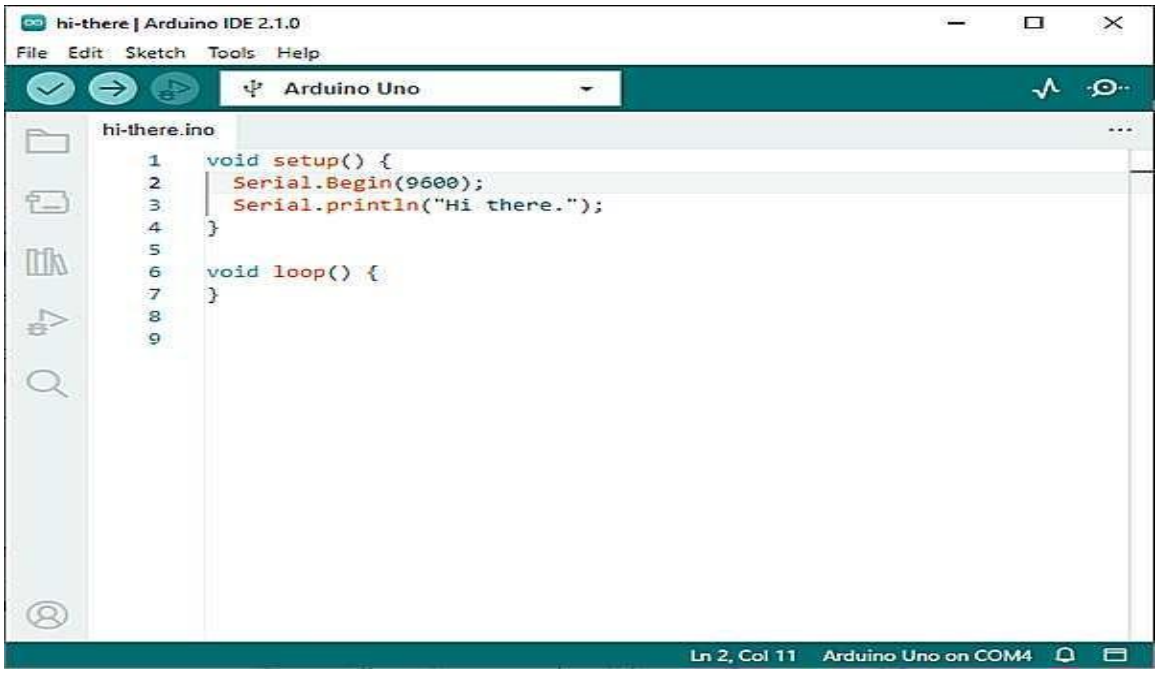
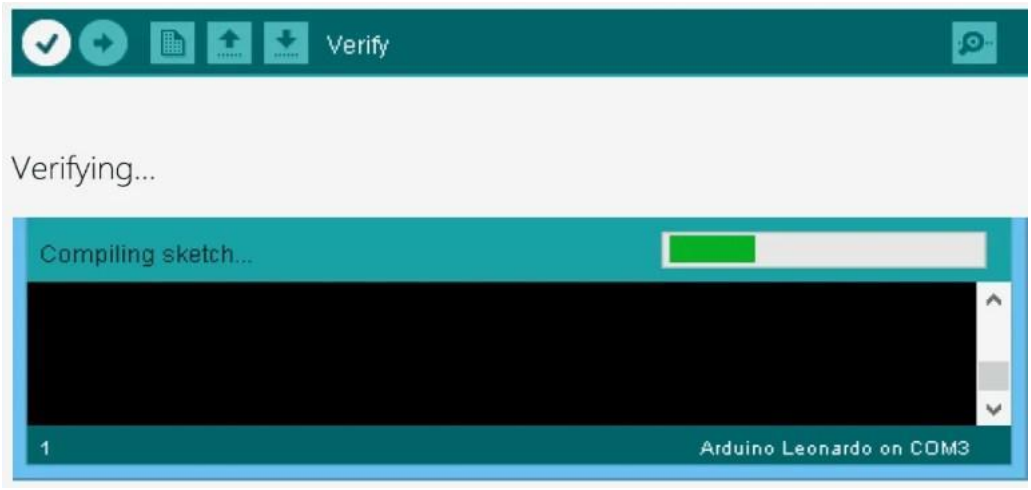


FIG 6.1.10 Arduino IDE Code Editor

Description:

14. Write program in Arduino IDE and include required libraries
15. Define pins and variables at the top of the sketch
16. Implement sensor data detection: ultrasonic water level, MQ135 gas sensor, flow sensor
17. Add water level calculation and threshold monitoring logic
18. Implement gas leakage detection using MQ135 sensor
19. Calculate flow rate using flow sensor interrupt
20. Add alert system using buzzer for local indication
21. Implement Telegram alert notification via WiFi
22. Add LCD display code for real-time monitoring

Step 5: Compiling the Code



Steps:

23. Click the Verify button (✓) in Arduino IDE toolbar

24. Check the output console for any errors

25. Fix all errors if present before uploading

Step 6: Uploading the Code to ESP32

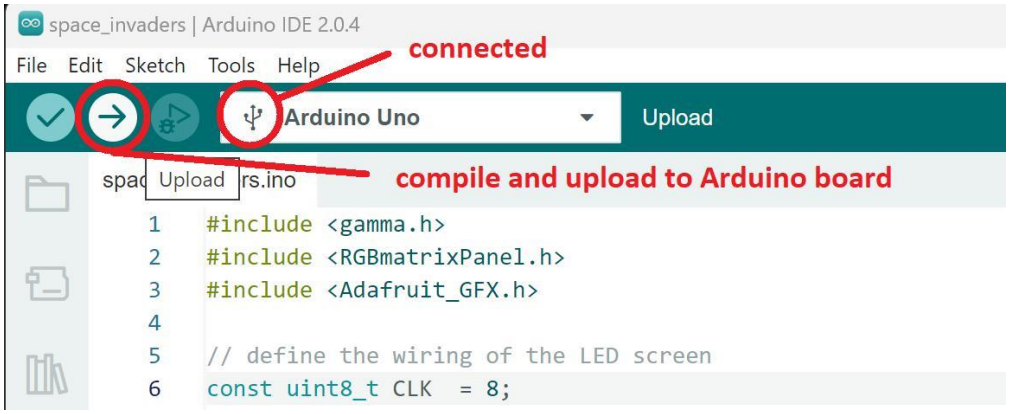


FIG 6.1.11 Uploading Code to ESP32

Steps:

26. Connect ESP32 to computer via USB cable

27. Select Board: Tools → Board → ESP32 Dev Module

28. Select Port: Tools → Port → COM port (e.g. COM3 on Windows)

29. Click the Upload button (→) to flash the code onto the ESP32

Step 7: Serial Monitor

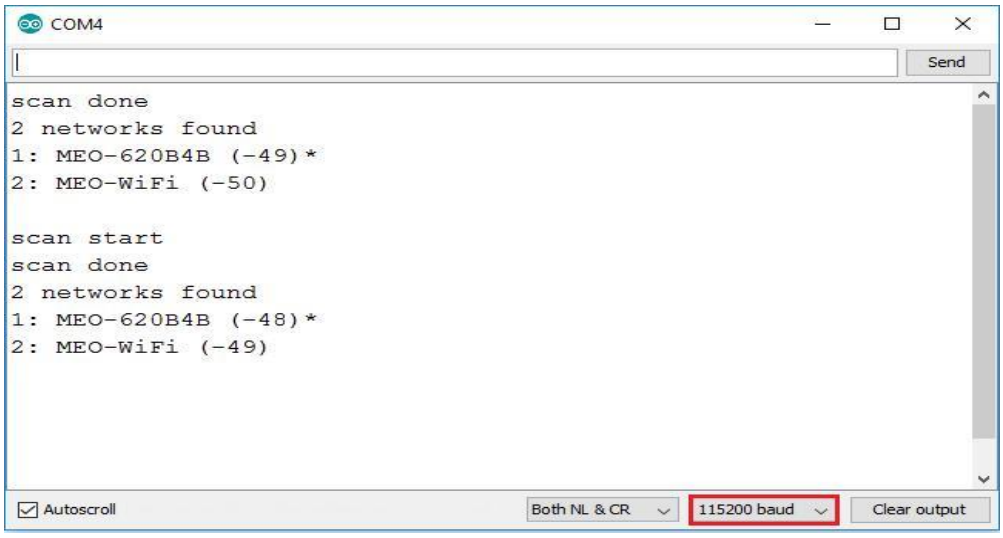


FIG 6.1.12 Serial Monitor Output

Steps:

30. Open Serial Monitor from Tools → Serial Monitor (or Ctrl+Shift+M)

31. Set baud rate to 115200 to match the code

Used for: Debugging sensor readings, checking WiFi connection status and monitoring output values.

Step 8: System Execution Flow

32. System initializes: ESP32, sensors, LCD display, and WiFi module

33. WiFi connection is established and Telegram bot starts

34. Sensors continuously monitor parameters in real time

35. Water level is measured using ultrasonic sensor

36. Gas level is detected using MQ135 sensor

37. Flow rate is calculated using flow sensor interrupt

38. System checks threshold conditions for water, gas, and flow

39. If abnormal condition detected, buzzer alert is triggered

40. Telegram message is sent via WiFi to the user's phone

41. Real-time data is displayed on the 16x2 LCD screen

CHAPTER-07
PROJECT RESULTS

CHAPTER-07

PROJECT RESULTS

7.1 RESULT

The EV battery monitoring system for early fire detection and safety successfully demonstrated reliable real-time monitoring of critical parameters such as temperature, gas leakage, and flame presence, ensuring timely identification of potential hazards. By integrating sensors like DHT11, MQ2 gas sensor, and flame sensor with the ESP32 microcontroller, the system was able to trigger immediate alerts through a buzzer and display warnings on the OLED screen whenever abnormal conditions were detected. The results showed that the system can effectively reduce the risk of thermal runaway and fire accidents by providing early warnings, thereby enhancing the overall safety of electric vehicles. Additionally, the compact design, low cost, and efficient performance make it a practical solution for implementation in EV battery management systems, contributing to improved reliability and user safety.

Performance analysis:

- The system demonstrated fast response time by detecting temperature rise, gas leakage, and flame presence almost instantly, enabling early warning of potential fire hazards.
- Sensors such as DHT11, MQ2, and the flame sensor showed satisfactory accuracy in identifying abnormal conditions when properly calibrated.
- The ESP32 microcontroller efficiently processed real-time data and triggered alerts (buzzer and OLED display) without noticeable delay.
- The system maintained stable and continuous operation with low power consumption, making it suitable for long-term monitoring in EVs.
- Overall, the system achieved reliable performance with minimal false alarms, effectively enhancing safety by preventing critical battery failure and fire incidents

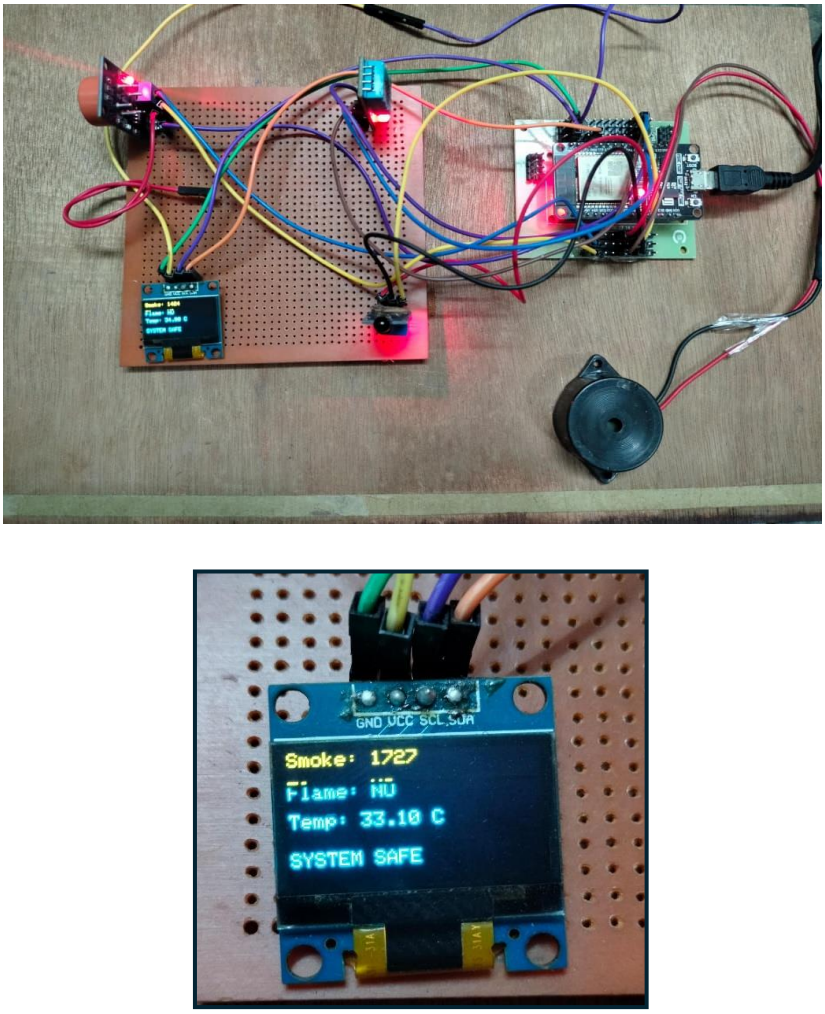


Fig 7.1. Initial State of The System

The above Fig shows the initial state of the EV battery monitoring system where it is indicated using a status LED. The status LED represents whether the system is powered and functioning properly. It remains ON when the system is active. The status LED is connected to the GPIO pin of the ESP32.

Indicates that:

• Power supply is available

• ESP32 microcontroller is running

• Program is executing successfully

• Sensor values are within safe limits

• No gas leakage or flame is detected

• Temperature is under normal condition

• No alert or buzzer is activated

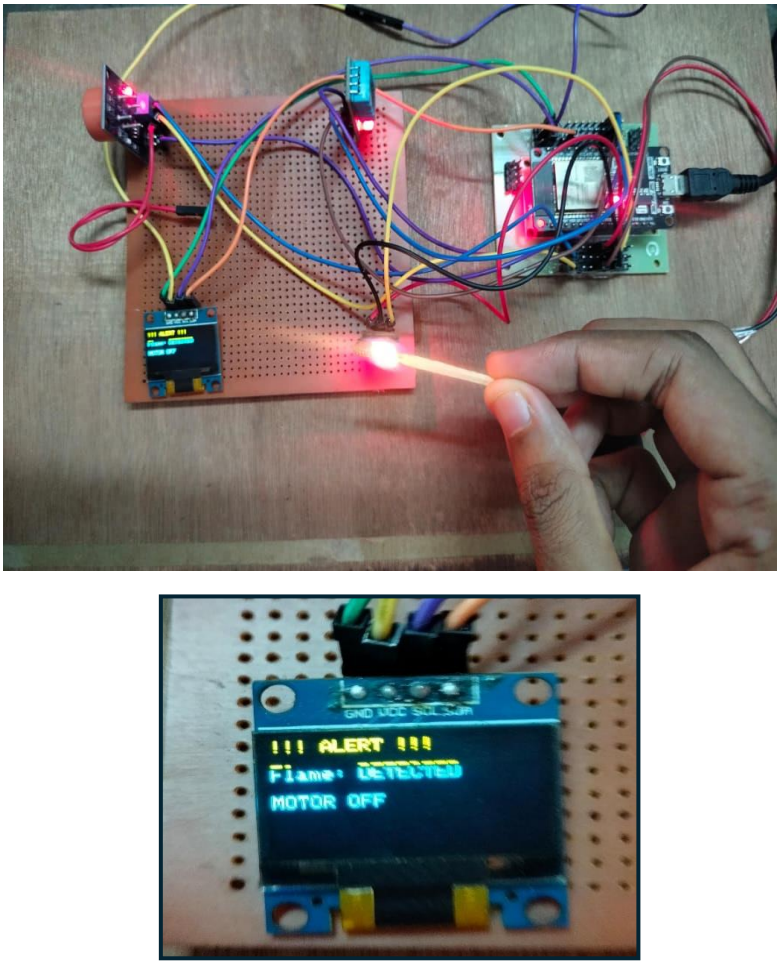


Fig 7.2. Flame Detection

The above Fig shows the flame detection state of the EV battery monitoring system where the presence of fire is identified using the flame sensor. When a flame source is detected near the sensor, the system immediately recognizes it as a potential hazard. The ESP32 processes this input and activates the alert mechanisms. The alert LED glows and the buzzer turns ON to indicate danger. The OLED display shows a warning message about flame detection. The flame sensor is connected to the GPIO pin of the ESP32.

Indicates that:

- Flame or fire is detected near the system
- ESP32 processes the signal and triggers alert
- Alert LED glows to indicate danger
- Buzzer is activated for audible warning
- Warning message is displayed on OLED
- Immediate action is required to prevent fire hazard

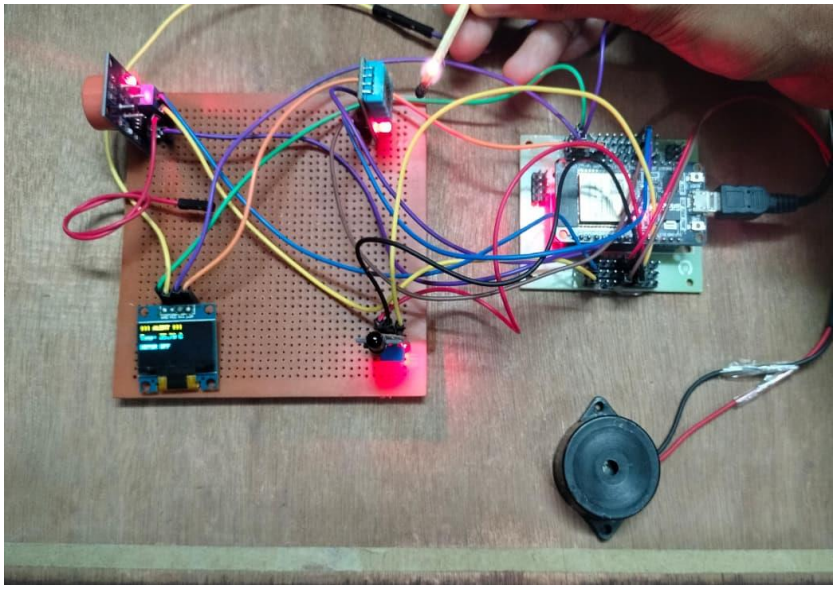


Fig 7.3. Battery Temperature detection

The above Fig shows the temperature detection state of the EV battery monitoring system where an increase in temperature is identified using the DHT11 sensor. When the temperature exceeds the predefined safe limit, the system recognizes it as a potential risk. The ESP32 processes this data and activates the alert system. The alert LED glows and the buzzer turns ON to indicate warning. The OLED display shows the temperature value along with a warning message.

Indicates that:

- Temperature exceeds safe limit
- ESP32 detects abnormal condition
- Alert LED is activated
- Buzzer gives warning sound
- Warning message displayed on OLED
- Indicates risk of overheating
- System responds to high temperature condition

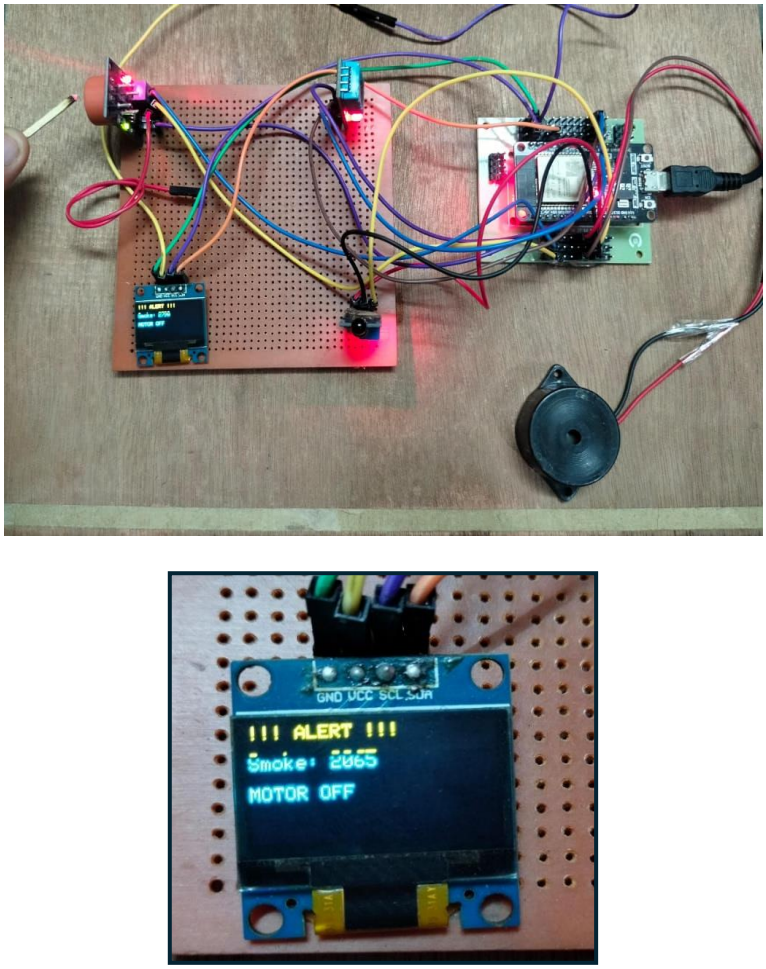


Fig 7.4. Smoke Detection

The above Fig shows the smoke (gas) detection state of the EV battery monitoring system where the presence of smoke or harmful gases is identified using the MQ2 gas sensor. When the gas level exceeds the predefined safe limit, the system detects it as a potential hazard. The ESP32 processes this signal and activates the alert system. The alert LED glows and the buzzer turns ON to indicate warning. The OLED display shows gas level along with an alert message. Indicates that:

- Smoke or gas is detected
- ESP32 identifies abnormal condition
- Alert LED is activated
- Buzzer gives warning sound
- Warning message displayed on OLED
- Indicates risk of fire or leakage
- System responds to gas detection condition

CHAPTER-08

ADVANTAGES AND APPLICATIONS

CHAPTER-08
ADVANTAGES AND APPLICATIONS

8.1 ADVANTAGES

Low Cost System

The proposed EV battery monitoring system is designed using cost-effective components such as ESP32, DHT11, MQ2 gas sensor, flame sensor, and OLED display. These components are easily available in the market at a low price, making the overall system affordable. This makes it highly suitable for student projects as well as practical implementations in electric vehicles without increasing the overall cost significantly.

Early Fire Detection

One of the major advantages of this system is its ability to detect fire hazards at an early stage. The flame sensor and gas sensor continuously monitor the battery environment for any signs of fire or harmful gas leakage. Early detection helps in taking preventive measures before the situation becomes dangerous, thus protecting both the vehicle and the user.

Real-Time Monitoring

The system continuously monitors important parameters such as temperature, gas levels, and fire presence. These values are updated in real time and displayed on the OLED screen. This ensures that any abnormal condition is immediately noticed, allowing quick action to prevent damage or failure.

Improved Safety

Safety is a critical factor in electric vehicles. This system improves safety by detecting overheating, gas leakage, or fire conditions in the battery. By providing timely alerts, it reduces the risk of accidents, battery explosions, and other hazardous situations.

Compact Design

The components used in this system are small in size and lightweight. This allows the system to be easily installed inside the EV battery compartment without occupying much space. Its compact design also makes it suitable for portable and embedded applications.

Energy Efficient

The system consumes very low power during operation. The ESP32 microcontroller and sensors are designed to work efficiently, making the system ideal for battery-powered applications. This ensures that the monitoring system does not significantly affect the overall battery performance.

User-Friendly Interface

The OLED display provides a simple and clear interface for users. It displays all important parameters such as temperature, gas levels, and fire status in an easy-to-read format. This helps users quickly understand the system condition without requiring technical knowledge.

Wireless Capability

The ESP32 microcontroller comes with built-in Wi-Fi and Bluetooth features. This allows the system to be upgraded in the future for wireless monitoring, mobile notifications, and IoT-based applications. It eliminates the need for additional communication modules.

Reliable Performance

The system provides stable and reliable performance over long periods. The sensors used are capable of detecting environmental changes accurately, ensuring proper monitoring of battery conditions under different situations.

Scalable System

The system can be easily expanded by adding more sensors or modules such as GPS, cloud connectivity, alarms, or advanced analytics. This flexibility allows future improvements without redesigning the entire system.

Reduced Human Effort

The monitoring process is automatic and does not require continuous human supervision. The system continuously checks for any abnormal conditions and provides alerts when needed, reducing manual effort and improving efficiency.

8.2 APPLICATIONS

Electric Vehicles (EVs)

The system is mainly used in electric vehicles to monitor battery conditions such as temperature, gas leakage, and fire hazards. It helps in preventing battery failures and ensures safe operation of EVs.

Battery Management Systems (BMS)

This project can be integrated into advanced battery management systems to enhance safety by providing real-time monitoring and early warning for abnormal conditions.

Energy Storage Systems

The system can be used in large-scale battery storage units such as solar power storage and backup power systems to detect overheating and fire risks.

Charging Stations

It can be implemented in EV charging stations to monitor battery conditions during charging and prevent overheating or short circuits.

Electric Scooters and Bikes

This system is useful in small electric vehicles like e-bikes and scooters where battery safety is critical due to compact design and high usage.

Industrial Battery Units

Industries that use large battery banks for machinery and backup systems can use this project to ensure safe operation and avoid accidents.

Portable Power Banks

The system can be applied in high-capacity power banks to monitor temperature and prevent overheating or explosion.

Smart Homes

It can be integrated into home battery backup systems (like inverters) to monitor battery health and detect fire risks.

Automotive Safety Systems

The project can be part of modern vehicle safety systems to provide alerts in case of battery malfunction or fire hazards.

IoT-Based Monitoring Systems

With ESP32, the system can be extended to IoT platforms for remote monitoring and alert notifications via mobile or cloud.

Research and Development

This system can be used for research purposes in improving battery safety technologies and developing advanced fire detection systems.

Educational Applications

It is highly useful for students to understand sensor integration, embedded systems, and real-time monitoring in EV technology.

CHAPTER-09

CONCLUSION &FUTURE SCOPE

9.1 CONCLUSION:

This project presents the design and implementation of an EV battery monitoring system aimed at early fire detection and enhanced safety. The system continuously monitors important parameters such as temperature, gas leakage, and flame presence using sensors like DHT11, MQ-2, and an IR flame sensor. These sensors help in detecting abnormal conditions such as overheating, release of flammable gases, or fire initiation inside the battery pack, which are common causes of thermal runaway in EV batteries.

The ESP32 microcontroller acts as the core of the system, collecting and processing sensor data in real time. Based on predefined threshold values, the system quickly identifies unsafe conditions and activates alerts through a buzzer. At the same time, the OLED display provides live status updates, ensuring that the user is always aware of the battery condition. The fast response time of the system plays a key role in preventing serious accidents by enabling early intervention.

In addition, the project emphasizes low power consumption, compact design, and cost-effectiveness, making it suitable for integration into real-world electric vehicles. The system is also flexible and can be further upgraded by adding features like wireless communication, cloud-based monitoring, and mobile notifications for remote access and control.

Overall, this project demonstrates how a simple yet intelligent monitoring system can significantly improve the safety and reliability of EV batteries. It not only reduces the risk of fire hazards but also increases user confidence in electric vehicles. With further enhancements and real-time data analytics, this system can contribute to the development of smarter and safer EV technologies in the future.

9.2 FUTURE SCOPE:

The EV battery monitoring system for early fire detection can be further enhanced in several ways to improve its efficiency, accuracy, and real-world applicability. One of the major future improvements is the integration of cloud technology and IoT platforms. By connecting the ESP32 to the internet, real-time data can be monitored remotely through mobile applications or web dashboards, allowing users and authorities to track battery health from anywhere.

Another important scope is the use of advanced and more accurate sensors. Replacing basic sensors like DHT11 with high-precision temperature and gas sensors can improve detection accuracy and response time. Additionally

REFERENCES:

1. S. Kumar, R. Sharma and P. Verma, "EV battery monitoring system for early fire detection and safety using IoT sensors," International Journal of Engineering Research & Technology (IJERT), vol. 11, no. 4, pp. 1021–1026, 2022.
2. A. Gupta, M. Singh and R. Patel, "Fire detection and prevention in electric vehicle battery systems using embedded controllers," International Journal of Advanced Research in Computer Science, vol. 12, no. 2, pp. 45–50, 2021.
3. P. Reddy, S. Kumar and N. Jain, "Design and implementation of battery thermal monitoring for EV safety applications," IEEE International Conference on Smart Computing and Systems Engineering, pp. 120–125, 2020.
4. V. Sharma, K. Mishra and R. Das, "Gas leakage detection in EV battery monitoring systems using MQ sensors," International Journal of Computer Applications, vol. 183, no. 15, pp. 12–16, 2021.
5. M. Rahman, S. Islam and M. Hossain, "Wireless sensor based early warning system for electric vehicle battery hazards," Wireless Personal Communications (Springer), vol. 110, no. 3, pp. 1567–1580, 2020.
6. K. Patel, D. Shah and R. Patel, "IoT based EV battery protection and monitoring system using ESP32," International Conference on Communication and Signal Processing, pp. 987–991, 2019.
7. N. Gupta and S. Sharma, "Smart EV battery monitoring and fire alert system for vehicle safety," International Journal of Innovative Technology and Exploring Engineering, vol. 9, no. 6, pp. 234–238, 2021.
8. R. Mehta, A. Joshi and T. Singh, "Early warning system for EV battery overheating using gas, flame and temperature sensors," International Journal of Scientific Research in Engineering and Management, vol. 10, no. 8, pp. 310–315, 2022.

APPENDIX

SOURCE CODE

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Sensors
#define MQ_PIN 34
#define FLAME_DO 27
#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// Actuators
#define RELAY_PIN 23
#define BUZZER_PIN 25

// Thresholds (tune these)
int smokeThreshold = 2000;
float tempThreshold = 33.0;

void setup() {
  Serial.begin(115200);

  pinMode(FLAME_DO, INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
```

```
  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  dht.begin();

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found");
    while (true);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
}

void loop() {

  // ===== READ SENSORS =====
  int smokeValue = analogRead(MQ_PIN);
  int flameState = digitalRead(FLAME_DO); // LOW = flame
  float temperature = dht.readTemperature();

  if (isnan(temperature)) {
    Serial.println("DHT Error");
    return;
  }

  // ===== PRINT NORMAL VALUES =====
  Serial.print("Smoke: ");
  Serial.print(smokeValue);
  Serial.print(" | Flame: ");
  Serial.print(flameState);
  Serial.print(" | Temp: ");
  Serial.println(temperature);
```

```
// ===== CHECK CONDITIONS =====
bool smokeAlert = smokeValue > smokeThreshold;
bool flameAlert = (flameState == LOW);
bool tempAlert  = temperature > tempThreshold;

bool anyAlert = smokeAlert || flameAlert || tempAlert;

// ===== OLED NORMAL MODE =====
if (!anyAlert) {
  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  display.clearDisplay();

  display.setCursor(0, 0);
  display.print("Smoke: ");
  display.println(smokeValue);

  display.setCursor(0, 15);
  display.print("Flame: ");
  display.println(flameAlert ? "YES" : "NO");

  display.setCursor(0, 30);
  display.print("Temp: ");
  display.print(temperature);
  display.println(" C");

  display.setCursor(0, 50);
  display.println("SYSTEM SAFE");

  display.display();
}

// ===== ALERT MODE =====
```

```
else {
  digitalWrite(RELAY_PIN, HIGH);   // MOTOR OFF
  digitalWrite(BUZZER_PIN, HIGH);

  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("!!! ALERT !!!");

  int y = 15;

  if (smokeAlert) {
    display.setCursor(0, y);
    display.print("Smoke: ");
    display.println(smokeValue);
    y += 12;
  }

  if (flameAlert) {
    display.setCursor(0, y);
    display.println("Flame: DETECTED");
    y += 12;
  }

  if (tempAlert) {
    display.setCursor(0, y);
    display.print("Temp: ");
    display.print(temperature);
    display.println(" C");
    y += 12;
  }

  display.setCursor(0, y + 5);
  display.println("MOTOR OFF");

  display.display();
```

```
    Serial.println("⚠ ALERT TRIGGERED → MOTOR OFF");
  }

  delay(1000);
}
```